

BANKING AND CUSTOMER 360 PLATFORM USING SNOWFLAKE

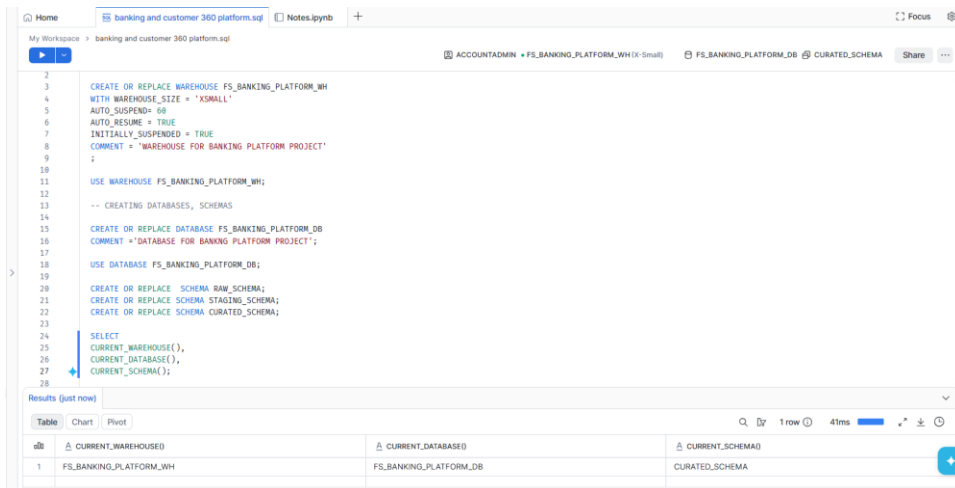
Objective :

To build an end-to-end banking data platform using Snowflake for data ingestion, data quality, transformation, automation, security, analytics, AI-based insights, and dashboard reporting.

Work Flow:

SOURCE DATA → INTERNAL STAGE → RAW LAYER → DATA VALIDATION → STAGING LAYER → CLEANING & STANDARDIZATION → CURATED LAYER → FACT & DIMENSION TABLES → STREAMS & TASKS → SECURITY → VIEWS & UDFs → CORTEX ANALYST → STREAMLIT DASHBOARD

1. WAREHOUSE, DATABASE, SCHEMAS

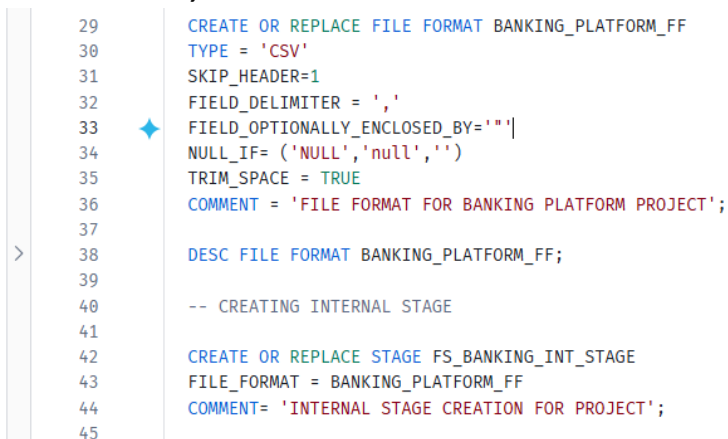


```
2
3 CREATE OR REPLACE WAREHOUSE FS_BANKING_PLATFORM_WH
4 WITH WAREHOUSE_SIZE = 'XSMALL'
5 AUTO_SUSPEND= 60
6 AUTO_RESUME = TRUE
7 INITIALLY_SUSPENDED = TRUE
8 COMMENT = 'WAREHOUSE FOR BANKING PLATFORM PROJECT'
9 ;
10
11 USE WAREHOUSE FS_BANKING_PLATFORM_WH;
12
13 -- CREATING DATABASES, SCHEMAS
14
15 CREATE OR REPLACE DATABASE FS_BANKING_PLATFORM_DB
16 COMMENT = 'DATABASE FOR BANKING PLATFORM PROJECT';
17
18 USE DATABASE FS_BANKING_PLATFORM_DB;
19
20 CREATE OR REPLACE SCHEMA RAW_SCHEMA;
21 CREATE OR REPLACE SCHEMA STAGING_SCHEMA;
22 CREATE OR REPLACE SCHEMA CURATED_SCHEMA;
23
24 SELECT
25     CURRENT_WAREHOUSE(),
26     CURRENT_DATABASE(),
27     CURRENT_SCHEMA();
28
```

Results (just now)

1	CURRENT_WAREHOUSE()	CURRENT_DATABASE()	CURRENT_SCHEMA()
1	FS_BANKING_PLATFORM_WH	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA

2. FILE FORMAT, STAGE



```
29 CREATE OR REPLACE FILE FORMAT BANKING_PLATFORM_FF
30 TYPE = 'CSV'
31 SKIP_HEADER=1
32 FIELD_DELIMITER = ','
33 FIELD_OPTIONALLY_ENCLOSED_BY='"'
34 NULL_IF= ('NULL','null','')
35 TRIM_SPACE = TRUE
36 COMMENT = 'FILE FORMAT FOR BANKING PLATFORM PROJECT';
37
38 DESC FILE FORMAT BANKING_PLATFORM_FF;
39
40 -- CREATING INTERNAL STAGE
41
42 CREATE OR REPLACE STAGE FS_BANKING_INT_STAGE
43 FILE_FORMAT = BANKING_PLATFORM_FF
44 COMMENT= 'INTERNAL STAGE CREATION FOR PROJECT';
45
```

3. CREATING RAW TABLES IN RAW SCHEMA

```
48 -- CREATING RAW TABLES
49
50 CREATE OR REPLACE TABLE RAW_CUSTOMERS (
51   customer_id VARCHAR,first_name VARCHAR, last_name VARCHAR,
52   gender VARCHAR,dob VARCHAR, email VARCHAR, phone VARCHAR, address VARCHAR,
53   city VARCHAR, state VARCHAR, postal_code VARCHAR,
54   registration_date VARCHAR, is_active VARCHAR, customer_segment VARCHAR, occupation VARCHAR
55 );
56
57
58 CREATE OR REPLACE TABLE RAW_ACCOUNTS (
59
60   account_id VARCHAR, customer_id VARCHAR, branch_id VARCHAR, account_type VARCHAR, account_number VARCHAR, current_balance VARCHAR, opening_date VARCHAR, status VARCHAR,
61   account_tier VARCHAR
62 );
63
64 CREATE OR REPLACE TABLE RAW_BRANCHES(
65
66   branch_id VARCHAR, branch_code VARCHAR, branch_name VARCHAR, city VARCHAR, state VARCHAR,
67   branch_type VARCHAR, email VARCHAR, phone VARCHAR, status VARCHAR
68 );
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
CREATE OR REPLACE TABLE RAW_CARDS(
  card_id VARCHAR, customer_id VARCHAR, account_id VARCHAR, card_type VARCHAR, masked_card_number VARCHAR, issue_date VARCHAR, expiry_date VARCHAR, status VARCHAR,
  card_variant VARCHAR
);

CREATE OR REPLACE TABLE RAW_PAYMENTS (
  payment_id VARCHAR, transaction_id VARCHAR,customer_id VARCHAR, payment_date VARCHAR,payment_amount VARCHAR,
  payment_method VARCHAR, payment_status VARCHAR, merchant_id VARCHAR,payment_channel VARCHAR
)

CREATE OR REPLACE TABLE RAW_TRANSACTIONS (
  transaction_id VARCHAR, account_id VARCHAR, customer_id VARCHAR, transaction_timestamp VARCHAR, transaction_type VARCHAR,
  direction VARCHAR, amount VARCHAR, currency VARCHAR, status VARCHAR, channel_ VARCHAR,
  merchant_name VARCHAR, reference_number VARCHAR
);

SHOW TABLES IN SCHEMA RAW_SCHEMA;
```

id	created_on	name	database_name	schema_name	kind	comment	cluster_by	rows	bytes	owner
1	2026-09-27 05:02:09.523 -0700	RAW_ACCOUNTS	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			8000	299520	ACCOUNTADMIN
2	2026-09-27 05:03:34.666 -0700	RAW_BRANCHES	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			50	3584	ACCOUNTADMIN
3	2026-09-27 05:05:06.086 -0700	RAW_CARDS	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			5000	174080	ACCOUNTADMIN
4	2026-09-27 05:00:03.835 -0700	RAW_CUSTOMERS	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			14980	778240	ACCOUNTADMIN
5	2026-09-27 05:06:40.270 -0700	RAW_PAYMENTS	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			25000	887296	ACCOUNTADMIN
6	2026-09-27 05:08:57.704 -0700	RAW_TRANSACTIONS	FS_BANKING_PLATFORM_DB	RAW_SCHEMA	TABLE			49900	2258432	ACCOUNTADMIN

4. USING SNOWSQL UPLOADING FILES INTO SNOWFLAKE

```
Microsoft Windows [Version 10.0.26280.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sridatta>snowsql -v
Version: 1.5.1

C:\Users\sridatta>snowsql -a GQZHFJE-0277403 -u AKAGAMI18
Password:
* SnowSQL * v1.5.1
Type SQL statements or !help
AKAGAMI18#COMPUTE_WH(no database).(no schema)>USE DATABASE
FS_BANKING_PLATFORM_DB;

status
Statement executed successfully.
1 Row(s) produced. Time Elapsed: 0.092s
AKAGAMI18#COMPUTE_WH@FS_BANKING_PLATFORM_DB.PUBLIC>USE WAREHOUSE FS_BANKING_PLATFORM_WH;

status
Statement executed successfully.
1 Row(s) produced. Time Elapsed: 0.113s
AKAGAMI18#FS_BANKING_PLATFORM_WH@FS_BANKING_PLATFORM_DB.PUBLIC>USE DATABASE FS_BANKING_PLATFORM_DB;

status
Statement executed successfully.
1 Row(s) produced. Time Elapsed: 0.088s
AKAGAMI18#FS_BANKING_PLATFORM_WH@FS_BANKING_PLATFORM_DB.PUBLIC>USE SCHEMA RAW_SCHEMA;

status
Statement executed successfully.
1 Row(s) produced. Time Elapsed: 0.092s
AKAGAMI18#FS_BANKING_PLATFORM_WH@FS_BANKING_PLATFORM_DB.RAW_SCHEMA>PUT FILE://
```

```
AKAGAMI18#FS_BANKING_PLATFORM_WH@FS_BANKING_PLATFORM_DB.RAW_SCHEMA>PUT 'FILE:///C:/Users/sridatta/OneDrive/Desktop/snowflake project/*.csv'
@FS_BANKING_INT_STAGE AUTO_COMPRESS = TRUE;
```

source	target	source_size	target_size	source_compression	target_compression	status	message
accounts.csv	accounts.csv.gz	670314	202672	NONE	GZIP	UPLOADED	
branches.csv	branches.csv.gz	5842	1152	NONE	GZIP	UPLOADED	
cards.csv	cards.csv.gz	497873	100912	NONE	GZIP	UPLOADED	
customers.csv	customers.csv.gz	834143	193360	NONE	GZIP	UPLOADED	
payments.csv	payments.csv.gz	2286349	582576	NONE	GZIP	UPLOADED	
transactions.csv	transactions.csv.gz	6051714	1543904	NONE	GZIP	UPLOADED	

```
6 Row(s) produced. Time Elapsed: 2.55s
AKAGAMI18#FS_BANKING_PLATFORM_WH@FS_BANKING_PLATFORM_DB.RAW_SCHEMA>
```

5. USING COPY INTO COMMAND TO INSERT DATA INTO TABLES

```
-- LOADING DATA INTO TABLES FROM STAGE

COPY INTO RAW_ACCOUNTS FROM @FS_BANKING_INT_STAGE PATTERN = '.*accounts.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
COPY INTO RAW_BRANCHES FROM @FS_BANKING_INT_STAGE PATTERN = '.*branches.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
COPY INTO RAW_CARDS FROM @FS_BANKING_INT_STAGE PATTERN = '.*cards.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
COPY INTO RAW_CUSTOMERS FROM @FS_BANKING_INT_STAGE PATTERN = '.*customers.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
COPY INTO RAW_PAYMENTS FROM @FS_BANKING_INT_STAGE PATTERN = '.*payments.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
COPY INTO RAW_TRANSACTIONS FROM @FS_BANKING_INT_STAGE PATTERN = '.*transactions.*' FILE_FORMAT = (FORMAT_NAME = BANKING_PLATFORM_FF) ON_ERROR = CONTINUE;
```

```
SELECT 'RAW_CUSTOMERS' AS TABLE_, COUNT(*) AS ROW_COUNT FROM RAW_CUSTOMERS
UNION ALL
SELECT 'RAW_ACCOUNTS', COUNT(*) FROM RAW_ACCOUNTS
UNION ALL
SELECT 'RAW_BRANCHES', COUNT(*) FROM RAW_BRANCHES
UNION ALL
SELECT 'RAW_CARDS', COUNT(*) FROM RAW_CARDS
UNION ALL
SELECT 'RAW_PAYMENTS', COUNT(*) FROM RAW_PAYMENTS
UNION ALL
SELECT 'RAW_TRANSACTIONS', COUNT(*) FROM RAW_TRANSACTIONS ;
```

Results (just now)

TABLE_	ROW_COUNT
RAW_CUSTOMERS	14980
RAW_ACCOUNTS	8000
RAW_BRANCHES	50
RAW_CARDS	5000
RAW_PAYMENTS	25000
RAW_TRANSACTIONS	49900

6. DATA VALIDATION AND CLEANING AND CREATING STAGING TABLES

CUSTOMERS TABLE :

```
SELECT * FROM RAW_CUSTOMERS;
```

```
SELECT COUNT(*) AS TOTAL_ROWS FROM RAW_CUSTOMERS;
```

```
SELECT
```

```
COUNT(*) AS TOTAL_ROWS,
COUNT_IF(TRIM(CUSTOMER_ID)='') AS NULL_CID,
COUNT_IF(TRIM(DOB)='') AS NULL_DOB,
COUNT_IF(TRIM(EMAIL)='') AS NULL_EMAIL,
COUNT_IF(TRIM(PHONE)='') AS NULL_PHONE,
COUNT_IF(TRIM(CITY)='') AS NULL_CITY,
COUNT_IF(TRIM(POSTAL_CODE)='') AS NULL_POSTAL_CODE,
```

```
FROM RAW_CUSTOMERS;
```

```
SELECT CUSTOMER_ID, COUNT(*) AS TOTAL_COUNT FROM
RAW_CUSTOMERS GROUP BY CUSTOMER_ID HAVING TOTAL_COUNT >1
ORDER BY TOTAL_COUNT DESC;
```

```
SELECT *
FROM RAW_CUSTOMERS
WHERE CUSTOMER_ID IN (
SELECT CUSTOMER_ID
FROM (
SELECT CUSTOMER_ID, ROW_NUMBER() OVER ( PARTITION BY
CUSTOMER_ID ORDER BY CUSTOMER_ID) AS RN
FROM RAW_CUSTOMERS
)
WHERE RN > 1
);
```

```
-- DATA CLEANING
```

```
SELECT * FROM RAW_CUSTOMERS;
```

```
INSERT INTO RAW_CUSTOMERS
SELECT *
FROM RAW_SCHEMA.RAW_CUSTOMERS
AT (OFFSET => -60*60*1);
```

```
UPDATE RAW_CUSTOMERS SET CUSTOMER_ID =
TRIM(UPPER(CUSTOMER_ID));
--ALTER RAW_CUSTOMERS ADD_COLUMN FULL_NAME =
TRIM(CONCAT(TRIM(FIRST_NAME), ' ', TRIM(LAST_NAME))
SELECT DISTINCT(GENDER) FROM RAW_CUSTOMERS;
```

```
SELECT * FROM RAW_CUSTOMERS WHERE DOB IS NULL;
SELECT * FROM RAW_CUSTOMERS WHERE EMAIL IS NULL;
SELECT * FROM RAW_CUSTOMERS WHERE PHONE IS NULL;
SELECT DISTINCT CITY, POSTAL_CODE FROM RAW_CUSTOMERS;
SELECT * FROM RAW_CUSTOMERS WHERE POSTAL_CODE IS NULL;
SELECT * FROM RAW_CUSTOMERS WHERE REGISTRATION_DATE IS NULL;
SELECT DISTINCT CUSTOMER_SEGMENT FROM RAW_CUSTOMERS;
SELECT DISTINCT IS_ACTIVE FROM RAW_CUSTOMERS;
```

```
-- CHHANGING DATA TYPES
```

```
UPDATE RAW_CUSTOMERS SET DOB = REPLACE(DOB, '/', '-');
```

```
UPDATE RAW_CUSTOMERS AS T
SET POSTAL_CODE = N.POSTAL_CODE FROM RAW_CUSTOMERS N
WHERE T.CITY = N.CITY AND T.POSTAL_CODE IS NULL AND
N.POSTAL_CODE IS NOT NULL;
```

```
-- CREATING STAGING TABLE
```

```
CREATE OR REPLACE TABLE
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_CUSTOMERS
AS
SELECT
CUSTOMER_ID :: VARCHAR AS CUSTOMER_ID,
FIRST_NAME :: VARCHAR AS FIRST_NAME,
LAST_NAME :: VARCHAR AS LAST_NAME,
CONCAT(FIRST_NAME,' ',LAST_NAME) :: VARCHAR AS FULL_NAME,
GENDER :: VARCHAR AS GENDER,
TRY_TO_DATE(DOB) AS DATE_OF_BIRTH,
EMAIL :: VARCHAR AS EMAIL,
PHONE :: VARCHAR AS PHONE,
ADDRESS :: VARCHAR AS ADDRESS,
CITY :: VARCHAR AS CITY,
STATE :: VARCHAR AS STATE,
POSTAL_CODE:: VARCHAR AS POSTAL_CODE,
TRY_TO_TIMESTAMP(REGISTRATION_DATE) AS REGISTRATION_DATE,
CASE
    WHEN UPPER(TRIM(IS_ACTIVE)) IN ('TRUE', 'YES', '1', 'Y', 'ACTIVE')
        THEN TRUE
    WHEN UPPER(TRIM(IS_ACTIVE)) IN ('FALSE', 'NO', '0', 'N', 'INACTIVE')
        THEN FALSE
    ELSE NULL
END AS IS_ACTIVE,
CUSTOMER_SEGMENT :: VARCHAR AS CUSTOMER_SEGMENT,
OCCUPATION :: VARCHAR AS OCCUPATION
FROM FS_BANKING_PLATFORM_DB.RAW_SCHEMA.RAW_CUSTOMERS;
```

```
UPDATE STAGING_SCHEMA.STAGING_CUSTOMERS
SET
CUSTOMER_ID = TRIM(UPPER(CUSTOMER_ID)),
FIRST_NAME = TRIM(INITCAP(FIRST_NAME)),
LAST_NAME = TRIM(INITCAP(LAST_NAME)),
GENDER = TRIM(INITCAP(GENDER)),
EMAIL = TRIM(LOWER(EMAIL)),
CITY = TRIM(INITCAP(CITY)),
STATE = TRIM(INITCAP(STATE)),
IS_ACTIVE = TRIM(INITCAP(IS_ACTIVE));
```

```
DELETE FROM STAGING_SCHEMA.STAGING_CUSTOMERS
WHERE CUSTOMER_ID IN (
SELECT CUSTOMER_ID FROM (
SELECT CUSTOMER_ID,ROW_NUMBER() OVER
(PARTITION BY CUSTOMER_ID ORDER BY CUSTOMER_ID)
AS R_N
FROM STAGING_SCHEMA.STAGING_CUSTOMERS
)
WHERE R_N > 1
);
```

```
SELECT * FROM STAGING_CUSTOMERS;
```

ACCOUNTS TABLE :

```
SELECT * FROM RAW_ACCOUNTS;
```

```
SELECT ACCOUNT_ID,COUNT(*) FROM RAW_ACCOUNTS GROUP BY
ACCOUNT_ID ORDER BY COUNT(*) DESC;
```

```
SELECT * FROM RAW_ACCOUNTS WHERE ACCOUNT_ID IN (
SELECT ACCOUNT_ID FROM(
SELECT ACCOUNT_ID, ROW_NUMBER() OVER(PARTITION BY
ACCOUNT_ID ORDER BY ACCOUNT_ID) AS R_N FROM RAW_ACCOUNTS
) WHERE R_N >1
);
```

```
SELECT * FROM RAW_ACCOUNTS WHERE ACCOUNT_NUMBER IS NULL;
```

```
SELECT DISTINCT STATUS FROM RAW_ACCOUNTS;
SELECT ACCOUNT_NUMBER, COUNT(*) FROM RAW_ACCOUNTS GROUP
BY ACCOUNT_NUMBER ORDER BY COUNT(*) DESC;
```

```
SELECT DISTINCT BRANCH_ID FROM RAW_ACCOUNTS;
```

```
-- CREATING STAGING TABLE
```

```
CREATE OR REPLACE TABLE
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_ACCOUNTS
AS
SELECT
ACCOUNT_ID :: VARCHAR AS ACCOUNT_ID,
CUSTOMER_ID :: VARCHAR AS CUSTOMER_ID,
BRANCH_ID :: VARCHAR AS BRANCH_ID,
ACCOUNT_TYPE :: VARCHAR AS ACCOUNT_TYPE,
ACCOUNT_NUMBER :: VARCHAR AS ACCOUNT_NUMBER,
TRY_TO_DECIMAL(CURRENT_BALANCE) AS CURRENT_BALANCE,
TRY_TO_DATE(OPENING_DATE) AS OPENING_DATE,
STATUS :: VARCHAR AS STATUS,
ACCOUNT_TIER :: VARCHAR AS ACCOUNT_TIER
FROM RAW_ACCOUNTS;
```

```
ALTER TABLE STAGING_ACCOUNTS ADD COLUMN BALANCE_QUALITY
VARCHAR;
UPDATE STAGING_ACCOUNTS
SET BALANCE_QUALITY =
CASE
    WHEN CURRENT_BALANCE IS NULL THEN 'MISSING'
    WHEN CURRENT_BALANCE < 0 THEN 'INVALID'
    ELSE 'VALID'
END;
```

```
ALTER TABLE STAGING_ACCOUNTS DROP COLUMN AMOUNT_QUALITY;
```

```
UPDATE STAGING_SCHEMA.STAGING_ACCOUNTS
SET
ACCOUNT_ID = TRIM(UPPER(ACCOUNT_ID)),
CUSTOMER_ID = TRIM(UPPER(CUSTOMER_ID)),
BRANCH_ID = TRIM(BRANCH_ID),
ACCOUNT_TYPE = TRIM(UPPER(ACCOUNT_TYPE)),
ACCOUNT_NUMBER = TRIM(UPPER(ACCOUNT_NUMBER)),
CURRENT_BALANCE = TRIM(CURRENT_BALANCE),
OPENING_DATE = TRIM(REPLACE(OPENING_DATE, '/', '-')),
STATUS = TRIM(INITCAP(STATUS)),
ACCOUNT_TIER = TRIM(INITCAP(ACCOUNT_TIER));
```

```
SELECT * FROM STAGING_ACCOUNTS;
```

BRANCHES TABLE:

```
SELECT * FROM RAW_BRANCHES;
```

```
SELECT BRANCH_ID, COUNT(*) FROM RAW_BRANCHES GROUP BY
BRANCH_ID ORDER BY COUNT(*) DESC;
```

```
SELECT BRANCH_CODE, COUNT(*) FROM RAW_BRANCHES GROUP BY
BRANCH_CODE ORDER BY COUNT(*) DESC;
```

```
SELECT DISTINCT CITY FROM RAW_BRANCHES;
SELECT DISTINCT STATE FROM RAW_BRANCHES;
SELECT DISTINCT BRANCH_TYPE FROM RAW_BRANCHES;
```

```
SELECT * FROM RAW_BRANCHES WHERE EMAIL NOT LIKE '%@%';
SELECT * FROM RAW_BRANCHES WHERE PHONE NOT LIKE '%+91%';
SELECT DISTINCT STATUS FROM RAW_BRANCHES;
```

```
SELECT DISTINCT BRANCH_NAME FROM RAW_BRANCHES WHERE
BRANCH_NAME LIKE '%Mumbai%';
```

```
ALTER TABLE RAW_BRANCHES ADD COLUMN BRANCH_CATEGORY
VARCHAR;
```

```
UPDATE RAW_BRANCHES
SET BRANCH_CATEGORY =
CASE
    WHEN UPPER(TRIM(BRANCH_NAME)) LIKE '% MAIN' THEN 'Main'
    WHEN UPPER(TRIM(BRANCH_NAME)) LIKE '% CITY' THEN 'City'
    WHEN UPPER(TRIM(BRANCH_NAME)) LIKE '% MARKET' THEN 'Market'
    WHEN UPPER(TRIM(BRANCH_NAME)) LIKE '% CENTRAL' THEN 'Central'
    ELSE 'Unknown'
END;
```

```

CREATE OR REPLACE TABLE
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_BRANCHES AS
SELECT
BRANCH_ID :: VARCHAR AS BRANCH_ID,
BRANCH_CODE :: VARCHAR AS BRANCH_CODE,
BRANCH_NAME :: VARCHAR AS BRANCH_NAME,
CITY :: VARCHAR AS CITY,
STATE :: VARCHAR AS STATE,
BRANCH_TYPE :: VARCHAR AS BRANCH_TYPE,
BRANCH_CATEGORY :: VARCHAR AS BRANCH_CATEGORY,
EMAIL :: VARCHAR AS EMAIL,
PHONE :: VARCHAR AS PHONE,
STATUS :: VARCHAR AS STATUS
FROM RAW_BRANCHES;

```

```

UPDATE STAGING_SCHEMA.STAGING_BRANCHES
SET
BRANCH_ID = TRIM(BRANCH_ID),
BRANCH_CODE = TRIM(UPPER(BRANCH_CODE)),
BRANCH_NAME = TRIM(INITCAP(REPLACE(BRANCH_NAME, 'Branch', ''))),
CITY = TRIM(INITCAP(CITY)),
STATE = TRIM(INITCAP(STATE)),
BRANCH_TYPE = TRIM(INITCAP(BRANCH_TYPE)),
EMAIL = TRIM(LOWER(EMAIL)),
PHONE = REPLACE(PHONE, '+91-', ''),
STATUS = TRIM(INITCAP(STATUS));

```

CARDS TABLE :

```
SELECT * FROM RAW_CARDS;
```

```

SELECT CARD_ID, COUNT(*) FROM RAW_CARDS GROUP BY CARD_ID
ORDER BY COUNT(*) DESC;
SELECT CUSTOMER_ID, COUNT(*) FROM RAW_CARDS GROUP BY
CUSTOMER_ID ORDER BY COUNT(*) DESC;
SELECT ACCOUNT_ID, COUNT(*) FROM RAW_CARDS GROUP BY
ACCOUNT_ID ORDER BY COUNT(*) DESC;
SELECT DISTINCT CARD_TYPE FROM RAW_CARDS;
SELECT * FROM RAW_CARDS WHERE MASKED_CARD_NUMBER IS NULL;
SELECT * FROM RAW_CARDS WHERE ISSUE_DATE > EXPIRY_DATE;
SELECT DISTINCT STATUS FROM RAW_CARDS;
SELECT DISTINCT CARD_VARIANT FROM RAW_CARDS;

```

```

--CREATING STAGING TABLE
CREATE OR REPLACE TABLE
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_CARDS AS
SELECT
CARD_ID :: VARCHAR AS CARD_ID,
CUSTOMER_ID :: VARCHAR AS CUSTOMER_ID,
ACCOUNT_ID :: VARCHAR AS ACCOUNT_ID,
CARD_TYPE :: VARCHAR AS CARD_TYPE,
MASKED_CARD_NUMBER :: VARCHAR AS MASKED_CARD_NUMBER,
TRY_TO_DATE(ISSUE_DATE) AS ISSUE_DATE,
TRY_TO_DATE(EXPIRY_DATE) AS EXPIRY_DATE,
STATUS :: VARCHAR AS STATUS,
CARD_VARIANT :: VARCHAR AS CARD_VARIANT
FROM RAW_CARDS;

```

```

UPDATE STAGING_SCHEMA.STAGING_CARDS
SET CARD_TYPE =
CASE
WHEN TRIM(UPPER(CARD_TYPE)) IN ('CREDIT CARD', 'CREDIT')
THEN 'Credit Card'

```

```

WHEN TRIM(UPPER(CARD_TYPE)) IN ('DEBIT', 'DEBIT CARD')
THEN 'Debit Card'

```

```

WHEN TRIM(UPPER(CARD_TYPE)) IN ('PREPAID', 'PRE-PAID')
THEN 'Pre-Paid'

```

```

ELSE 'Unknown'
END;

```

```

UPDATE STAGING_SCHEMA.STAGING_CARDS
SET
EXPIRY_DATE = ISSUE_DATE, ISSUE_DATE = '2024-01-01'
WHERE ISSUE_DATE > EXPIRY_DATE;

```

```

UPDATE STAGING_SCHEMA.STAGING_CARDS
SET
CARD_ID = TRIM(UPPER(CARD_ID)),
CUSTOMER_ID = TRIM(UPPER(CUSTOMER_ID)),
ACCOUNT_ID = TRIM(UPPER(ACCOUNT_ID)),
CARD_TYPE = TRIM(INITCAP(CARD_TYPE)),
MASKED_CARD_NUMBER = TRIM(UPPER(MASKED_CARD_NUMBER)),
ISSUE_DATE = TRIM(REPLACE(ISSUE_DATE, '/', '-')),
EXPIRY_DATE = TRIM(REPLACE(EXPIRY_DATE, '/', '-')),
STATUS = TRIM(INITCAP(STATUS)),

CARD_VARIANT = TRIM(INITCAP(CARD_VARIANT));

```

PAYMENTS TABLE :

```
SELECT * FROM RAW_PAYMENTS;
```

```

SELECT PAYMENT_ID, COUNT(*) FROM RAW_PAYMENTS GROUP BY
PAYMENT_ID ORDER BY COUNT(*) DESC;
SELECT * FROM RAW_PAYMENTS WHERE TRANSACTION_ID IS NULL;
SELECT * FROM RAW_PAYMENTS WHERE CUSTOMER_ID IS NULL;
SELECT * FROM RAW_PAYMENTS WHERE PAYMENT_DATE IS NULL;
SELECT * FROM RAW_PAYMENTS WHERE PAYMENT_STATUS IS NULL;
SELECT * FROM RAW_PAYMENTS WHERE MERCHANT_ID IS NULL;
SELECT DISTINCT PAYMENT_METHOD FROM RAW_PAYMENTS;
SELECT DISTINCT PAYMENT_STATUS FROM RAW_PAYMENTS;
SELECT DISTINCT PAYMENT_CHANNEL FROM RAW_PAYMENTS;

```

```
SELECT * FROM RAW_PAYMENTS;
```

```
-- CREATING STAGING PAYMENTS TABLE
```

```

CREATE OR REPLACE TABLE
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_PAYMENTS AS
SELECT
PAYMENT_ID :: VARCHAR AS PAYMENT_ID,
TRANSACTION_ID :: VARCHAR AS TRANSACTION_ID,
CUSTOMER_ID :: VARCHAR AS CUSTOMER_ID,
TRY_TO_DATE(PAYMENT_DATE) AS PAYMENT_DATE,
TRY_TO_DECIMAL(PAYMENT_AMOUNT) AS PAYMENT_AMOUNT,
PAYMENT_METHOD :: VARCHAR AS PAYMENT_METHOD,
PAYMENT_STATUS :: VARCHAR AS PAYMENT_STATUS,
MERCHANT_ID :: VARCHAR AS MERCHANT_ID,
PAYMENT_CHANNEL :: VARCHAR AS PAYMENT_CHANNEL
FROM RAW_PAYMENTS;

```

```

ALTER TABLE STAGING_PAYMENTS ADD COLUMN
PAYMENT_AMOUNT_QUALITY VARCHAR;

```

```

UPDATE STAGING_PAYMENTS
SET PAYMENT_AMOUNT_QUALITY =
CASE
WHEN PAYMENT_AMOUNT IS NULL THEN 'MISSING'
WHEN PAYMENT_AMOUNT < 0 THEN 'INVALID'
ELSE 'VALID'
END;

```

```

UPDATE STAGING_SCHEMA.STAGING_PAYMENTS
SET PAYMENT_METHOD =
CASE
WHEN TRIM(UPPER(PAYMENT_METHOD)) = 'WALLET'
THEN 'Wallet'

```

```

WHEN TRIM(UPPER(PAYMENT_METHOD)) IN ('CREDIT_CARD', 'CREDIT
CARD')
THEN 'Credit Card'

```

```

WHEN TRIM(UPPER(PAYMENT_METHOD)) = 'UPI'
THEN 'Upi'

```

```

WHEN TRIM(UPPER(PAYMENT_METHOD)) = 'CASH'
THEN 'Cash'

```

```

WHEN TRIM(UPPER(PAYMENT_METHOD)) IN ('NETBANKING', 'NET
BANKING')

```

```
THEN 'Net-Banking'
```

```
ELSE 'Not Mentioned'  
END;
```

```
UPDATE STAGING_SCHEMA.STAGING_PAYMENTS  
SET  
PAYMENT_ID = TRIM(UPPER(PAYMENT_ID)),  
TRANSACTION_ID = TRIM(UPPER(TRANSACTION_ID)),  
CUSTOMER_ID = TRIM(UPPER(CUSTOMER_ID)),  
PAYMENT_DATE = TRIM(UPPER(PAYMENT_DATE)),  
PAYMENT_AMOUNT = TRIM(UPPER(PAYMENT_AMOUNT)),  
PAYMENT_METHOD = TRIM(UPPER(PAYMENT_METHOD)),  
PAYMENT_STATUS = TRIM(UPPER(PAYMENT_STATUS)),  
MERCHANT_ID = TRIM(UPPER(MERCHANT_ID)),  
PAYMENT_CHANNEL = TRIM(UPPER(PAYMENT_CHANNEL));
```

TRANSACTIONS TABLE:

```
SELECT * FROM RAW_TRANSACTIONS;  
SELECT * FROM RAW_TRANSACTIONS WHERE ACCOUNT_ID IS NULL;  
SELECT * FROM RAW_TRANSACTIONS WHERE CUSTOMER_ID IS NULL;  
SELECT * FROM RAW_TRANSACTIONS WHERE  
TRANSACTION_TIMESTAMP IS NULL;  
SELECT DISTINCT TRANSACTION_TYPE FROM RAW_TRANSACTIONS  
ORDER BY TRANSACTION_TYPE;  
SELECT DISTINCT DIRECTION FROM RAW_TRANSACTIONS;  
SELECT * FROM RAW_TRANSACTIONS WHERE AMOUNT IS NULL;  
SELECT DISTINCT CURRENCY FROM RAW_TRANSACTIONS;  
SELECT DISTINCT STATUS FROM RAW_TRANSACTIONS;  
SELECT DISTINCT CHANNEL_ FROM RAW_TRANSACTIONS;  
SELECT DISTINCT MERCHANT_NAME FROM RAW_TRANSACTIONS ORDER  
BY MERCHANT_NAME;
```

```
SELECT TRANSACTION_ID, COUNT(*) FROM RAW_TRANSACTIONS  
GROUP BY TRANSACTION_ID ORDER BY COUNT(*) DESC;
```

```
SELECT COUNT(*) FROM RAW_TRANSACTIONS WHERE TRANSACTION_ID  
IN (  
SELECT TRANSACTION_ID FROM (  
SELECT TRANSACTION_ID, ROW_NUMBER() OVER(PARTITION BY  
TRANSACTION_ID ORDER BY TRANSACTION_ID) AS R_N FROM  
RAW_TRANSACTIONS  
) WHERE R_N >1  
)  
;
```

```
DELETE FROM RAW_TRANSACTIONS WHERE TRANSACTION_ID IN (  
SELECT TRANSACTION_ID FROM (  
SELECT TRANSACTION_ID, ROW_NUMBER() OVER(PARTITION BY  
TRANSACTION_ID ORDER BY TRANSACTION_ID) AS R_N FROM  
RAW_TRANSACTIONS  
) WHERE R_N >1  
)  
;
```

```
UPDATE RAW_TRANSACTIONS SET  
TRANSACTION_TYPE = TRIM(INITCAP(TRANSACTION_TYPE));
```

```
UPDATE RAW_TRANSACTIONS SET  
STATUS =  
CASE  
WHEN TRIM(UPPER(STATUS)) IN ('FAILED', 'FAIL') THEN 'Failed'
```

```
ELSE TRIM(INITCAP(STATUS))  
END;
```

```
CREATE OR REPLACE TABLE  
FS_BANKING_PLATFORM_DB.STAGING_SCHEMA.STAGING_TRANSACTION  
S AS  
SELECT  
TRIM(UPPER(TRANSACTION_ID)) :: VARCHAR AS TRANSACTION_ID,  
TRIM(UPPER(ACCOUNT_ID)) :: VARCHAR AS ACCOUNT_ID,  
TRIM(UPPER(CUSTOMER_ID)) :: VARCHAR AS CUSTOMER_ID,  
TRY_TO_TIMESTAMP(TRANSACTION_TIMESTAMP) AS  
TRANSACTION_TIMESTAMP,  
TRANSACTION_TYPE :: VARCHAR AS TRANSACTION_TYPE,  
TRIM(UPPER(DIRECTION)) :: VARCHAR AS DIRECTION,  
TRY_TO_DECIMAL(AMOUNT) AS AMOUNT,  
TRIM(UPPER(CURRENCY)) :: VARCHAR AS CURRENCY,  
TRIM(UPPER(STATUS)) :: VARCHAR AS STATUS,  
TRIM(UPPER(CHANNEL_)) :: VARCHAR AS CHANNEL_,  
TRIM(UPPER(MERCHANT_NAME)) :: VARCHAR AS MERCHANT_NAME,  
TRIM(UPPER(REFERENCE_NUMBER)) :: VARCHAR AS  
REFERENCE_NUMBER
```

```
FROM RAW_TRANSACTIONS;
```

```
ALTER TABLE STAGING_TRANSACTIONS  
ADD COLUMN AMOUNT_QUALITY VARCHAR;
```

```
UPDATE STAGING_TRANSACTIONS  
SET AMOUNT_QUALITY =  
CASE  
WHEN AMOUNT IS NULL THEN 'MISSING'  
WHEN AMOUNT < 0 THEN 'INVALID'  
ELSE 'VALID'  
END;
```

```
UPDATE STAGING_SCHEMA.STAGING_TRANSACTIONS  
SET TRANSACTION_TYPE = TRIM(INITCAP(TRANSACTION_TYPE));
```

```
UPDATE STAGING_SCHEMA.STAGING_TRANSACTIONS  
SET STATUS =  
CASE  
WHEN TRIM(UPPER(STATUS)) IN ('FAILED', 'FAIL')  
THEN 'Failed'  
ELSE TRIM(INITCAP(STATUS))  
END;
```

591
592 SHOW TABLES IN SCHEMA STAGING_SCHEMA;
593

Results (just now)

Table Chart Pivot

id	created_on	name	database_name	schema_name
1	2026-09-27 23:09:13.611 -0700	STAGING_ACCOUNTS	FS_BANKING_PLATFORM_DB	STAGING_SCI
2	2026-09-28 04:13:38.011 -0700	STAGING_BRANCHES	FS_BANKING_PLATFORM_DB	STAGING_SCI
3	2026-09-28 00:09:39.601 -0700	STAGING_CARDS	FS_BANKING_PLATFORM_DB	STAGING_SCI
4	2026-09-27 22:40:31.547 -0700	STAGING_CUSTOMERS	FS_BANKING_PLATFORM_DB	STAGING_SCI
5	2026-09-28 04:11:05.190 -0700	STAGING_PAYMENTS	FS_BANKING_PLATFORM_DB	STAGING_SCI
6	2026-09-28 05:05:09.528 -0700	STAGING_TRANSACTIONS	FS_BANKING_PLATFORM_DB	STAGING_SCI

-- DATA VALIDATION IN STAGING TABLES

```
SELECT COUNT(*) FROM STAGING_ACCOUNTS WHERE CURRENT_BALANCE IS NULL OR CURRENT_BALANCE <0;
SELECT COUNT(*) FROM STAGING_PAYMENTS WHERE PAYMENT_AMOUNT IS NULL OR PAYMENT_AMOUNT <0;
SELECT COUNT(*) FROM STAGING_TRANSACTIONS WHERE AMOUNT IS NULL OR AMOUNT <0;
```

ESTABLISHING CURATED LAYER: FACT AND DIMENSION TABLES

-- DIMENSION TABLE CUSTOMERS

```
CREATE OR REPLACE TABLE
CURATED_SCHEMA.DIM_CUSTOMER AS
SELECT
CUSTOMER_ID, FIRST_NAME, LAST_NAME, GENDER,
DATE_OF_BIRTH, EMAIL, PHONE, ADDRESS,
CITY, STATE, POSTAL_CODE, REGISTRATION_DATE, IS_ACTIVE,
CUSTOMER_SEGMENT,
OCCUPATION
FROM STAGING_SCHEMA.STAGING_CUSTOMERS;
```

```
SELECT COUNT(*) FROM CURATED_SCHEMA.DIM_CUSTOMER;
```

-- DIMENSION TABLE ACCOUNTS

```
CREATE OR REPLACE TABLE CURATED_SCHEMA.DIM_ACCOUNT
AS
SELECT ACCOUNT_ID,CUSTOMER_ID,BRANCH_ID,
ACCOUNT_TYPE, ACCOUNT_NUMBER, CURRENT_BALANCE,
OPENING_DATE, STATUS, ACCOUNT_TIER
FROM STAGING_SCHEMA.STAGING_ACCOUNTS;
```

```
SELECT COUNT(*) FROM CURATED_SCHEMA.DIM_ACCOUNT;
```

-- CREATING DIMENSION TABLE BRANCHES

```
CREATE OR REPLACE TABLE CURATED_SCHEMA.DIM_BRANCH
AS
SELECT BRANCH_ID, BRANCH_CODE, BRANCH_NAME, CITY,
STATE, BRANCH_TYPE, EMAIL,
PHONE, STATUS
FROM STAGING_SCHEMA.STAGING_BRANCHES;
```

```
SELECT COUNT(*) FROM CURATED_SCHEMA.DIM_BRANCH;
```

-- CREATING DIMENSION TABLE CARDS

```
CREATE OR REPLACE TABLE CURATED_SCHEMA.DIM_CARD AS
SELECT CARD_ID, CUSTOMER_ID, ACCOUNT_ID, CARD_TYPE,
MASKED_CARD_NUMBER,
ISSUE_DATE, EXPIRY_DATE, STATUS, CARD_VARIANT
FROM STAGING_SCHEMA.STAGING_CARDS;
```

```
SELECT COUNT(*) FROM CURATED_SCHEMA.DIM_CARD;
SELECT COUNT(*) FROM STAGING_CARDS;
```

-- CREATING DIMENSION TABLE DATE

```
CREATE OR REPLACE TABLE CURATED_SCHEMA.DIM_DATE (
```

```
DATE_KEY INTEGER, FULL_DATE DATE, DAY_NUMBER INTEGER,
MONTH_NUMBER INTEGER, MONTH_NAME VARCHAR,
QUARTER_NUMBER INTEGER, YEAR_NUMBER INTEGER,
DAY_NAME VARCHAR, DAY_OF_WEEK INTEGER,
WEEK_OF_YEAR INTEGER
);
```

```
INSERT INTO CURATED_SCHEMA.DIM_DATE
SELECT
TO_NUMBER(TO_CHAR(DATE_VALUE, 'YYYYMMDD')) AS
DATE_KEY,
DATE_VALUE AS FULL_DATE,
DAY(DATE_VALUE) AS DAY_NUMBER,
MONTH(DATE_VALUE) AS MONTH_NUMBER,
MONTHNAME(DATE_VALUE) AS MONTH_NAME,
QUARTER(DATE_VALUE) AS QUARTER_NUMBER,
YEAR(DATE_VALUE) AS YEAR_NUMBER,
DAYNAME(DATE_VALUE) AS DAY_NAME,
DAYOFWEEK(DATE_VALUE) AS DAY_OF_WEEK,
WEEKOFYEAR(DATE_VALUE) AS WEEK_OF_YEAR
FROM (
SELECT DATEADD(
DAY,SEQ4(), '2020-01-01'::DATE
) AS DATE_VALUE
FROM TABLE(GENERATOR(ROWCOUNT => 3653))
);
```

-- CREATING FACT TABLE TRANSACTIONS

```
CREATE OR REPLACE TABLE
CURATED_SCHEMA.FACT_TRANSACTIONS AS
SELECT
TRANSACTION_ID, ACCOUNT_ID, CUSTOMER_ID,
CAST(TRANSACTION_TIMESTAMP AS DATE) AS DATE_KEY,
TRANSACTION_TIMESTAMP, TRANSACTION_TYPE, DIRECTION,
AMOUNT, CURRENCY, STATUS,
CHANNEL_, MERCHANT_NAME, REFERENCE_NUMBER
FROM STAGING_SCHEMA.STAGING_TRANSACTIONS;
```

```
CREATE OR REPLACE TABLE
CURATED_SCHEMA.FACT_PAYMENTS AS
SELECT PAYMENT_ID, TRANSACTION_ID, CUSTOMER_ID,
TO_NUMBER(TO_CHAR(PAYMENT_DATE, 'YYYYMMDD')) AS
DATE_KEY,
PAYMENT_DATE, PAYMENT_AMOUNT, PAYMENT_METHOD,
PAYMENT_STATUS, MERCHANT_ID,
PAYMENT_CHANNEL
FROM STAGING_SCHEMA.STAGING_PAYMENTS;
```

```
SELECT * FROM CURATED_SCHEMA.DIM_DATE;
```

-- Checking whether All Facts and Dimension Tables are correctly mapped or not

```
SELECT COUNT(*) FROM FACT_TRANSACTIONS T LEFT JOIN
DIM_CUSTOMER C ON T.CUSTOMER_ID = C.CUSTOMER_ID
WHERE C.CUSTOMER_ID IS NULL;
```

```
SELECT FT.CUSTOMER_ID FROM FACT_TRANSACTIONS FT
WHERE NOT EXISTS (
SELECT 1 FROM DIM_CUSTOMER DC WHERE DC.CUSTOMER_ID
= FT.CUSTOMER_ID
);
```

699 788 SHOW TABLES IN SCHEMA CURATED_SCHEMA;

Results (just now)

Table	Chart	Pivot	7 rows	109ms						
id	created_on	name	database_name	schema_name	kind	comment	cluster_by	# rows	# bytes	owner
1	2026-09-28 06:06:03.009 -0700	DIM_ACCOUNT	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			8000	243200	ACCOUNTAD
2	2026-09-28 06:07:48.458 -0700	DIM_BRANCH	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			50	3072	ACCOUNTAD
3	2026-09-28 06:09:08.290 -0700	DIM_CARD	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			5000	150016	ACCOUNTAD
4	2026-09-28 06:09:08.875 -0700	DIM_CUSTOMER	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			4980	258560	ACCOUNTAD
5	2026-09-28 06:11:23.406 -0700	DIM_DATE	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			3653	27136	ACCOUNTAD
6	2026-09-28 06:15:10.484 -0700	FACT_PAYMENTS	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			25000	750080	ACCOUNTAD
7	2026-09-28 06:14:05.558 -0700	FACT_TRANSACTIONS	FS_BANKING_PLATFORM_DB	CURATED_SCHEMA	TABLE			49900	2101760	ACCOUNTAD

CREATING VIEWS AND UDFS:

-- CREATING VIEWS

```
CREATE OR REPLACE VIEW CURATED_SCHEMA.CUST_TRANS_VIEW AS
SELECT
T.TRANSACTION_ID, T.TRANSACTION_TIMESTAMP, T.TRANSACTION_TYPE, T.DIRECTION, T.AMOUNT, T.CURRENCY,
T.STATUS, T.CHANNEL_, T.MERCHANT_NAME,
C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME, C.CUSTOMER_SEGMENT, C.CITY, C.STATE,
A.ACCOUNT_ID, A.ACCOUNT_TYPE, A.ACCOUNT_TIER
FROM
FACT_TRANSACTIONS T
LEFT JOIN DIM_CUSTOMER C ON T.CUSTOMER_ID = C.CUSTOMER_ID
LEFT JOIN DIM_ACCOUNT A ON A.ACCOUNT_ID = T.ACCOUNT_ID;
```

-- UDF

```
CREATE OR REPLACE FUNCTION CURATED_SCHEMA.CLASSIFY_TRANSACTION_AMOUNT( AMOUNT_VALUE NUMBER )
RETURNS VARCHAR
LANGUAGE SQL
AS
$$
CASE
WHEN AMOUNT_VALUE IS NULL THEN 'UNKNOWN'
WHEN AMOUNT_VALUE <= 1000 THEN 'LOW'
WHEN AMOUNT_VALUE <= 10000 THEN 'MEDIUM'
ELSE 'HIGH'
END
$$;
```

7. INCREMENTAL LOADING : CREATING STREAMS AND TASKS:

-- INCREMENTAL LOADING : STREAMS AND TASKS

```
CREATE OR REPLACE STREAM STAGING_TRANSACTION_STREAM ON TABLE STAGING_SCHEMA.STAGING_TRANSACTIONS
COMMENT = 'To capture changes in Transactons Table';
```

```
DESC STREAM STAGING_TRANSACTION_STREAM;
SELECT * FROM STAGING_TRANSACTION_STREAM;
```

-- task

```
CREATE OR REPLACE TASK STAGING_SCHEMA.LOAD_TNX_TASK
WAREHOUSE = FS_BANKING_PLATFORM_WH
SCHEDULE = '24 HOUR'
WHEN SYSTEM$$STREAM_HAS_DATA(
'STAGING_SCHEMA.STAGING_TRANSACTION_STREAM'
```

```

)
AS
INSERT INTO CURATED_SCHEMA.FACT_TRANSACTIONS(
TRANSACTION_ID, ACCOUNT_ID, CUSTOMER_ID, DATE_KEY, TRANSACTION_TIMESTAMP, TRANSACTION_TYPE, DIRECTION, AMOUNT,
CURRENCY, STATUS, CHANNEL_, MERCHANT_NAME, REFERENCE_NUMBER
)
SELECT
TRANSACTION_ID, ACCOUNT_ID, CUSTOMER_ID, TO_NUMBER(TO_CHAR(TRANSACTION_TIMESTAMP, 'YYYYMMDD')) AS DATE_KEY,
TRANSACTION_TIMESTAMP, TRANSACTION_TYPE, DIRECTION, AMOUNT, CURRENCY, STATUS, CHANNEL_, MERCHANT_NAME,
REFERENCE_NUMBER
FROM STAGING_SCHEMA.STAGING_TRANSACTION_STREAM
WHERE METADATA$ACTION = 'INSERT';

ALTER TASK STAGING_SCHEMA.LOAD_TNX_TASK RESUME;
ALTER TASK STAGING_SCHEMA.LOAD_TNX_TASK SUSPEND;

-- PAYMENTS STREAM

CREATE OR REPLACE STREAM STAGING_PAYMENTS_STREAM ON TABLE STAGING_SCHEMA.STAGING_PAYMENTS
COMMENT = 'To capture changes in Payments Table';

SELECT * FROM STAGING_SCHEMA.STAGING_PAYMENTS_STREAM;

-- TASK FOR PAYMENTS TABLE

CREATE OR REPLACE TASK STAGING_SCHEMA.LOAD_PAY_DATA
WAREHOUSE = FS_BANKING_PLATFORM_WH
SCHEDULE = '24 HOUR'
WHEN SYSTEM$$STREAM_HAS_DATA('STAGING_SCHEMA.STAGING_PAYMENTS_STREAM')
AS
INSERT INTO CURATED_SCHEMA.FACT_PAYMENTS (
PAYMENT_ID, TRANSACTION_ID, CUSTOMER_ID, DATE_KEY, PAYMENT_DATE, PAYMENT_AMOUNT, PAYMENT_METHOD,
PAYMENT_STATUS, MERCHANT_ID, PAYMENT_CHANNEL
)
SELECT
PAYMENT_ID, TRANSACTION_ID, CUSTOMER_ID, TO_NUMBER(TO_CHAR(PAYMENT_DATE, 'YYYYMMDD')) AS DATE_KEY,
PAYMENT_DATE, PAYMENT_AMOUNT, PAYMENT_METHOD, PAYMENT_STATUS, MERCHANT_ID, PAYMENT_CHANNEL
FROM STAGING_SCHEMA.STAGING_PAYMENTS_STREAM
WHERE METADATA$ACTION = 'INSERT';

ALTER TASK STAGING_SCHEMA.LOAD_PAY_DATA RESUME;
ALTER TASK STAGING_SCHEMA.LOAD_PAY_DATA SUSPEND;

```

8. APPLYING ROLE BASED ACCESS CONTROL:

```

CREATE ROLE BANKING_ADMIN;
CREATE ROLE BANKING_DEVELOPER;
CREATE ROLE BANKING_ANALYST;
CREATE ROLE BANKING_READONLY;

-- FOR BANKING ADMIN

GRANT USAGE ON DATABASE FS_BANKING_PLATFORM_DB TO
ROLE BANKING_ADMIN;
GRANT USAGE ON ALL SCHEMAS IN DATABASE
FS_BANKING_PLATFORM_DB TO ROLE BANKING_ADMIN;
GRANT ALL PRIVILEGES ON FUTURE TABLES IN DATABASE
FS_BANKING_PLATFORM_DB TO ROLE BANKING_ADMIN;

-- FOR BANKING DEVELOPER

GRANT USAGE ON DATABASE FS_BANKING_PLATFORM_DB TO
ROLE BANKING_DEVELOPER
GRANT USAGE ON SCHEMA RAW_SCHEMA TO ROLE
BANKING_DEVELOPER;
GRANT USAGE ON SCHEMA STAGING_SCHEMA TO ROLE
BANKING_DEVELOPER;
GRANT USAGE ON SCHEMA CURATED_SCHEMA TO ROLE
BANKING_DEVELOPER;
GRANT INSERT,UPDATE,DELETE,SELECT ON ALL TABLES IN
DATABASE FS_BANKING_PLATFORM_DB TO ROLE
BANKING_DEVELOPER;

-- FOR BANKING ANALYST

GRANT USAGE ON DATABASE FS_BANKING_PLATFORM_DB TO
ROLE BANKING_ANALYST;
GRANT USAGE ON SCHEMA CURATED_SCHEMA TO ROLE
BANKING_ANALYST;
GRANT SELECT ON ALL TABLES IN SCHEMA CURATED_SCHEMA
TO ROLE BANKING_ANALYST;

-- FOR BANKING READ ONLY

GRANT USAGE ON DATABASE FS_BANKING_PLATFORM_DB TO
ROLE BANKING_READONLY;
GRANT USAGE ON SCHEMA CURATED_SCHEMA TO ROLE
BANKING_READONLY;
GRANT SELECT ON ALL TABLES IN SCHEMA CURATED_SCHEMA
TO ROLE BANKING_READONLY;

-- ROLE HIERARCHY

-- Readonly → Analyst
GRANT ROLE BANKING_READONLY TO ROLE
BANKING_ANALYST;

-- Analyst → Developer

```

```
GRANT ROLE BANKING_ANALYST TO ROLE
BANKING_DEVELOPER;
```

```
-- Developer → Admin
GRANT ROLE BANKING_DEVELOPER TO ROLE BANKING_ADMIN;
```

```
-- Admin → Accountadmin
GRANT ROLE BANKING_ADMIN TO ROLE ACCOUNTADMIN;
```

```
-- CREATING USER AND PASSWORDS
```

```
CREATE USER BANK_DEV PASSWORD = 'P@ss123'
DEFAULT_ROLE = BANKING_DEVELOPER;
GRANT ROLE BANKING_DEVELOPER TO USER BANK_DEV;
```

```
CREATE USER BANK_ADM PASSWORD = 'Pa$ss123'
DEFAULT_ROLE = BANKING_ADMIN;
GRANT ROLE BANKING_ADMIN TO USER BANK_ADM;
```

```
CREATE USER BANK_AN PASSWORD = 'pAss123' DEFAULT_ROLE
= BANKING_ANALYST;
GRANT ROLE BANKING_ANALYST TO USER BANK_AN;
```

```
CREATE USER BANK_READ PASSWORD = 'PASS@123'
DEFAULT_ROLE = BANKING_READONLY;
GRANT ROLE BANKING_READONLY TO USER BANK_READ;
```

9. APPLYING DATA MASKING:

```
-- APPLYING DATA MASKING
```

```
CREATE OR REPLACE MASKING POLICY MASK_PHONE
AS (PHONE_VALUE VARCHAR)
RETURNS VARCHAR->
CASE WHEN CURRENT_ROLE() IN ('BANKING_ADMIN', 'BANKING_DEVELOPER') THEN PHONE_VALUE
ELSE 'XXXXXXXXXX' END;
```

```
ALTER TABLE CURATED_SCHEMA.DIM_CUSTOMER MODIFY COLUMN PHONE SET MASKING POLICY MASK_PHONE;
```

```
CREATE OR REPLACE MASKING POLICY MASK_EMAIL
AS (EMAIL_VALUE VARCHAR)
RETURNS VARCHAR->
CASE WHEN CURRENT_ROLE() IN ('BANKING_ADMIN', 'BANKING_DEVELOPER') THEN EMAIL_VALUE
ELSE 'xxxxx@xxxx.xxx' END;
```

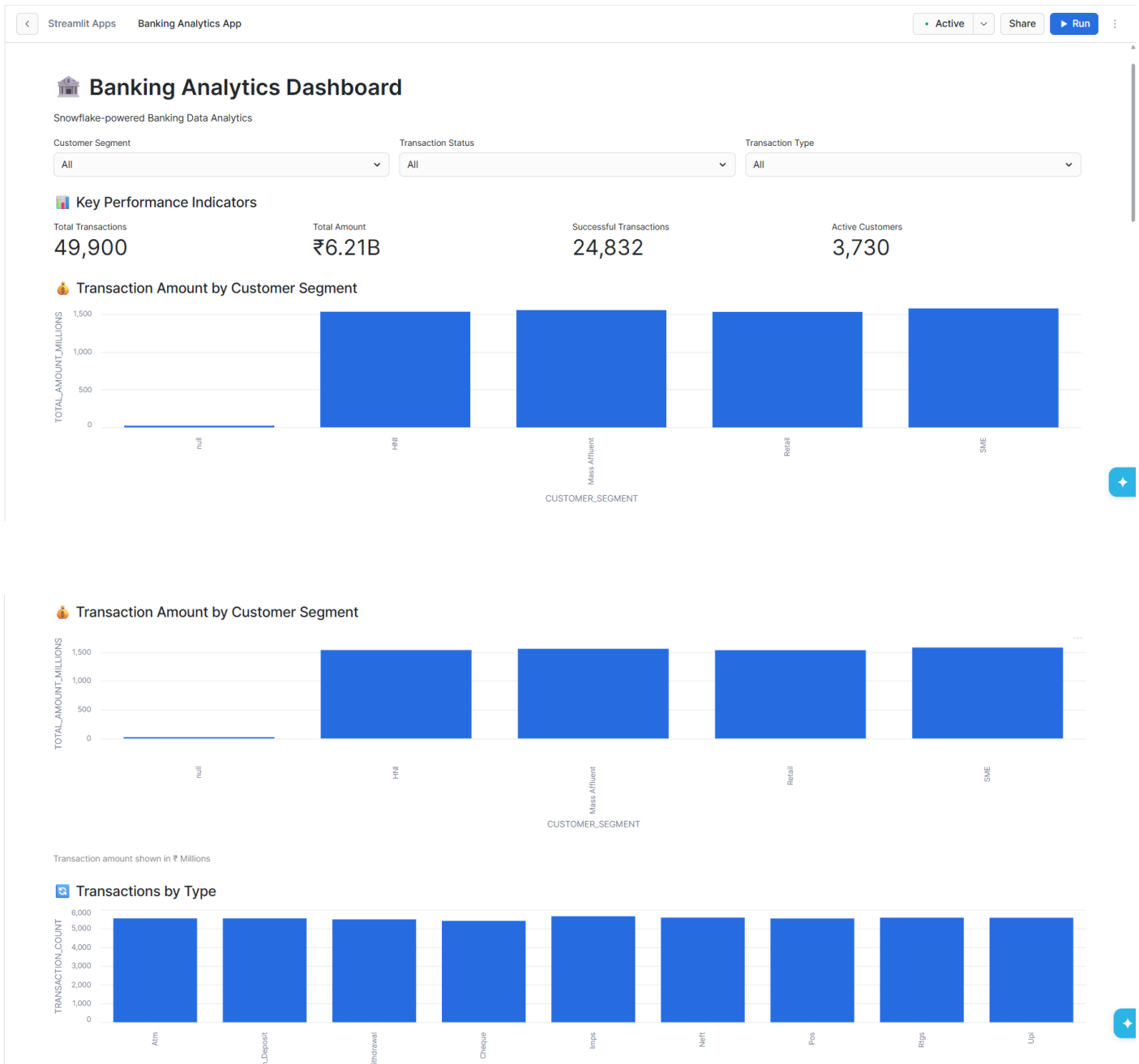
```
ALTER TABLE CURATED_SCHEMA.DIM_CUSTOMER MODIFY COLUMN EMAIL SET MASKING POLICY MASK_EMAIL;
```

10. CORTEX ANALYST:

The screenshot shows the Cortex Analyst interface with the 'Playground' tab selected. A user input 'what is the total transaction amount?' is shown in a blue box. Below it, the 'Cortex Analyst' section displays the interpretation: 'What is the total transaction amount over the entire available time period?'. A table shows the result: 'TOTAL_TRANSACTION_AMOUNT' with the value '6212817408'. Below the table, it says 'Only the first 10 rows are displayed as a preview'. A 'Semantic query' section shows the generated SQL query: 'SELECT * FROM SEMANTIC_VIEW(FS_BANKING_PLATFORM_DB.CURATED_SCHEMA.BANKING_ANALYTICS METRICS SUM(amount) AS total_transaction_amount) -- Generated by Cortex Analyst (request_id: a0ec48f1-4c83-4551-b8df-b3dec204c255) ;'. At the bottom, there is a button '+ Add to verified query'.

The screenshot shows the Cortex Analyst interface with the 'Playground' tab selected. A user input 'i need transaction type wise no.of transaction' is shown in a blue box. Below it, the 'Cortex Analyst' section displays the interpretation: 'What is the number of transactions for each transaction type?'. A table shows the results: 'NUMBER_OF_TRANSACTIONS' and 'TRANSACTION_TYPE'. The data rows are: 5464 Cash-Withdrawal, 5638 Imps, 5523 Cash-Deposit, 5550 Upi, 5523 Atm, 5558 Rtgs, 5520 Pos, 5560 Neft. Below the table, it says 'Only the first 10 rows are displayed as a preview'. A 'Semantic query' section shows the generated SQL query: 'SELECT * FROM SEMANTIC_VIEW(FS_BANKING_PLATFORM_DB.CURATED_SCHEMA.BANKING_ANALYTICS METRICS SUM(amount) AS total_transaction_amount) -- Generated by Cortex Analyst (request_id: a0ec48f1-4c83-4551-b8df-b3dec204c255) ;'. At the bottom, there is a button '+ Add to verified query'.

11. STREAMLIT APP : DASHBOARD



★ Transactions by Status



📄 Recent Transactions

	TRANSACTION_ID	TRANSACTION_TIMESTAMP	TRANSACTION_TYPE	AMOUNT	STATUS	CHANNEL	CUSTOMER_SEGMENT	CITY
0	TXN0000022852	None	Cheque	₹128.23K	Pending	Atm	Retail	Chennai
1	TXN0000031530	None	Rtgs	₹51.81K	Success	Atm	SME	Chennai
2	TXN0000035564	None	Rtgs	₹237.38K	Failed	Mobile App	Mass Affluent	Gurugram
3	TXN0000047135	None	Atm	₹40.51K	Success	Internet Banking	HNI	Jaipur
4	TXN0000031408	None	Cash_Deposit	₹87.34K	Success	Mobile App	HNI	Jaipur
5	TXN0000039929	None	Cash_Deposit	₹59.42K	Success	Apl	HNI	Noida
6	TXN0000018310	None	Atm	₹135.10K	Success	Pos	HNI	Mumbai
7	TXN0000013987	None	Imps	₹92.42K	Success	Branch	Retail	Pune
8	TXN0000018795	None	Imps	₹152.31K	Success	Mobile App	SME	Jaipur

Catalog explorer



▼ OZ77403

🔍 Find objects by name

📦 Data Products

▼ 🗄 Databases (5)

▼ 🗄 FS_BANKING_PLATFORM_DB (5)

▼ 🗄 CURATED_SCHEMA (10)

➤ Tables (7)

➤ Views (1)

➤ Semantic views (1)

➤ Functions (1)

➤ 🗄 INFORMATION_SCHEMA (63)

🗄 PUBLIC

▼ 🗄 RAW_SCHEMA (8)

➤ Tables (6)

➤ Stages (1)

➤ File formats (1)

▼ 🗄 STAGING_SCHEMA (10)

➤ Tables (6)

➤ Streams (2)

➤ Tasks (2)